

view index cascading

ea9dbb87-d02f-43b0-99da-b79b886df122.pdf

PDF

give me tables and answers in one sql server

```
SQL
-- =====
-- DATABASE CREATE
-- =====
CREATE DATABASE PracticeDB;
GO

USE PracticeDB;
GO

-- =====
-- TABLE CREATE
-- =====

CREATE TABLE Salesman(
    salesman_id INT PRIMARY KEY,
    name VARCHAR(50),
    city VARCHAR(50),
    commission DECIMAL(4,2)
);

CREATE TABLE Customer(
    customer_id INT PRIMARY KEY,
    cust_name VARCHAR(50),
    city VARCHAR(50),
    grade INT,
    salesman_id INT,
    FOREIGN KEY (salesman_id) REFERENCES Salesman(salesman_id)
);

CREATE TABLE Orders(
    ord_no INT PRIMARY KEY,
    purch_amt DECIMAL(10,2),
    ord_date DATE,
    customer_id INT,
    salesman_id INT,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (salesman_id) REFERENCES Salesman(salesman_id)
);

-- =====
-- INSERT DATA
-- =====

INSERT INTO Salesman VALUES
(5001,'James','New York',0.15),
(5002,'Nail','Paris',0.13),
(5005,'Pit','London',0.11),
(5006,'Mc Lyon','Paris',0.14),
(5007,'Paul','Rome',0.13);

INSERT INTO Customer VALUES
(3001,'Brad','New York',100,5001),
(3002,'Nick','Paris',200,5002),
(3003,'Alex','London',300,5005),
(3004,'John','Rome',300,5007),
(3005,'Miller','Paris',200,5006);

INSERT INTO Orders VALUES
(70001,1500,'2012-10-10',3001,5001),
(70002,2500,'2012-08-17',3002,5002),
(70003,3500,'2012-10-10',3003,5005),
(70004,4500,'2012-08-17',3004,5007),
(70005,5500,'2012-10-10',3005,5006);

-- =====
-- VIEW QUESTIONS ANSWERS
-- =====

-- 1
CREATE VIEW vw_newyork_salesman AS
SELECT *
FROM Salesman
WHERE city='New York';

-- 2
CREATE VIEW vw_salesman_basic AS
SELECT salesman_id,name,city
FROM Salesman;

-- 3
CREATE VIEW vw_newyork_commission AS
SELECT *
FROM Salesman
WHERE city='New York'
AND commission > 0.13;

-- 4
CREATE VIEW vw_customer_grade_count AS
SELECT grade,COUNT(*) AS total_customer
FROM Customer
GROUP BY grade;

-- 5
CREATE VIEW vw_order_summary AS
SELECT ord_date,
COUNT(DISTINCT customer_id) AS total_customer,
COUNT(DISTINCT salesman_id) AS total_salesman,
AVG(purch_amt) AS avg_order,
SUM(purch_amt) AS total_order
FROM Orders
GROUP BY ord_date;

-- 6
CREATE VIEW vw_order_details AS
SELECT o.ord_no,
s.name AS salesman_name,
c.cust_name AS customer_name
FROM Orders o
JOIN Salesman s ON o.salesman_id=s.salesman_id
JOIN Customer c ON o.customer_id=c.customer_id;

-- 7
CREATE VIEW vw_highest_order_salesman AS
SELECT s.name,c.cust_name,o.purch_amt,o.ord_date
FROM Orders o
JOIN Salesman s ON o.salesman_id=s.salesman_id
JOIN Customer c ON o.customer_id=c.customer_id
WHERE o.purch_amt=
(
SELECT MAX(purch_amt)
FROM Orders
);

-- 8
CREATE VIEW vw_salesman_highest3 AS
SELECT salesman_id,COUNT(*) AS total_highest
FROM Orders
GROUP BY salesman_id
HAVING COUNT(*)>=3;

-- 9
CREATE VIEW vw_highest_grade_customer AS
SELECT *
FROM Customer
```

```

WHERE grade=
(
SELECT MAX(grade)
FROM Customer
);

-- 10
CREATE VIEW vw_salesman_city_count AS
SELECT city,COUNT(*) AS total_salesman
FROM Salesman
GROUP BY city;

-- 11
CREATE VIEW vw_salesman_order_avg_total AS
SELECT s.name,
AVG(o.purch_amt) AS avg_order,
SUM(o.purch_amt) AS total_order
FROM Salesman s
JOIN Orders o
ON s.salesman_id=o.salesman_id
GROUP BY s.name;

-- 12
CREATE VIEW vw_salesman_more_customer AS
SELECT s.name,COUNT(c.customer_id) AS total_customer
FROM Salesman s
JOIN Customer c
ON s.salesman_id=c.salesman_id
GROUP BY s.name
HAVING COUNT(c.customer_id)>1;

-- 13
CREATE VIEW vw_customer_salesman_match AS
SELECT c.cust_name,
c.city AS customer_city,
s.name AS salesman_name,
s.city AS salesman_city
FROM Customer c
JOIN Salesman s
ON c.salesman_id=s.salesman_id;

-- 14
CREATE VIEW vw_orders_each_day AS
SELECT ord_date,
COUNT(*) AS total_orders
FROM Orders
GROUP BY ord_date;

-- 15
CREATE VIEW vw_salesman_oct10 AS
SELECT DISTINCT s.*
FROM Salesman s
JOIN Orders o
ON s.salesman_id=o.salesman_id
WHERE ord_date='2012-10-10';

-- 16
CREATE VIEW vw_salesman_aug_oct AS
SELECT DISTINCT s.*
FROM Salesman s
JOIN Orders o
ON s.salesman_id=o.salesman_id
WHERE ord_date='2012-08-17'
OR ord_date='2012-10-10';

-- =====
-- INDEXING QUESTIONS
-- =====

CREATE TABLE Worker(
    WORKER_ID INT PRIMARY KEY,
    FIRST_NAME VARCHAR(50),
    DEPARTMENT VARCHAR(50),
    SALARY INT
);

-- 1
CREATE INDEX idx_worker_salary
ON Worker(SALARY);

-- 2
CREATE INDEX idx_name_dept
ON Worker(FIRST_NAME,DEPARTMENT);

-- 3
CREATE UNIQUE INDEX idx_worker_id_unique
ON Worker(WORKER_ID);

-- 4
DROP INDEX idx_worker_salary
ON Worker;

-- =====
-- CASCADING QUESTIONS
-- =====

-- 1
CREATE TABLE Departments(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Staff(
    StaffID INT PRIMARY KEY,
    StaffName VARCHAR(50),
    DeptID INT,
    FOREIGN KEY (DeptID)
    REFERENCES Departments(DeptID)
    ON DELETE CASCADE
);

-- 2
CREATE TABLE Clients(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50)
);

ALTER TABLE Orders
ADD CONSTRAINT fk_cust_update
FOREIGN KEY (customer_id)
REFERENCES Clients(CustomerID)
ON UPDATE CASCADE;

-- 3
CREATE TABLE Projects(
    ProjectID INT,
    DeptCode VARCHAR(10),
    ProjectName VARCHAR(50),
    PRIMARY KEY(ProjectID,DeptCode)
);

CREATE TABLE Project_Tasks(
    TaskID INT PRIMARY KEY,
    ProjectID INT,
    DeptCode VARCHAR(10),
    TaskName VARCHAR(50),
    FOREIGN KEY(ProjectID,DeptCode)
    REFERENCES Projects(ProjectID,DeptCode)
    ON DELETE CASCADE
);

-- 4
CREATE TABLE Warehouse(
    WarehouseID INT PRIMARY KEY,
    WarehouseName VARCHAR(50)
);

CREATE TABLE Shipments(
    ShipmentID INT PRIMARY KEY,
    WarehouseID INT,

```

```

        ProductName VARCHAR(50),
        FOREIGN KEY (WarehouseID)
        REFERENCES Warehouse(WarehouseID)
        ON UPDATE CASCADE
        ON DELETE NO ACTION
    );

-- =====
-- VIEW OUTPUT CHECK
-- =====

SELECT * FROM vw_newyork_salesman;
SELECT * FROM vw_salesman_basic;
SELECT * FROM vw_customer_grade_count;
SELECT * FROM vw_order_summary;
SELECT * FROM vw_order_details;

```

SQL Server VIEW Full Details

What is a VIEW?

A **VIEW** in SQL Server is a **virtual table**. It does not store data permanently like a real table. Instead, it stores a SQL query. When you select data from a view, SQL Server runs the stored query and shows the result.

Syntax of VIEW

```

SQL
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;

```

Why We Use VIEW

1. Security

You can hide some columns from users. Example:

```

SQL
CREATE VIEW vw_employee AS
SELECT id, name
FROM Employee;

```

Users cannot see salary column.

2. Simplify Complex Queries

Instead of writing long JOIN queries repeatedly, create a view once.

3. Reusability

Use the same query many times.

4. Data Abstraction

Users see only necessary data.

Types of VIEW

Type	Description
Simple View	Uses one table
Complex View	Uses multiple tables, JOIN, GROUP BY
Indexed View	Stores data physically with index
System View	Built-in SQL Server views

1. Simple VIEW

Uses only one table.

Example Table

```

SQL
CREATE TABLE Student(
    id INT,
    name VARCHAR(50),
    department VARCHAR(50)
);

```

Create View

```

SQL
CREATE VIEW vw_student AS
SELECT id, name
FROM Student;

```

Use View

```

SQL
SELECT * FROM vw_student;

```

2. Complex VIEW

Uses JOIN, GROUP BY, Aggregate Functions.

Example

```
SQL
CREATE VIEW vw_order_details AS
SELECT c.cust_name,
o.purch_amt
FROM Customer c
JOIN Orders o
ON c.customer_id=o.customer_id;
```

How to See All Views

```
SQL
SELECT * FROM sys.views;
```

How to See VIEW Query

```
SQL
sp_helptext 'vw_student';
```

ALTER VIEW

Used to modify existing view.

Syntax

```
SQL
ALTER VIEW view_name AS
SELECT column_name
FROM table_name;
```

Example

```
SQL
ALTER VIEW vw_student AS
SELECT id,name,department
FROM Student;
```

DROP VIEW

Delete a view.

Syntax

```
SQL
DROP VIEW view_name;
```

Example

```
SQL
DROP VIEW vw_student;
```

INSERT INTO VIEW

Possible only if view is simple.

Example

```
SQL
CREATE VIEW vw_simple AS
SELECT id,name
FROM Student;
```

```
SQL
INSERT INTO vw_simple
VALUES(1,'Rahim');
```

UPDATE VIEW

```
SQL
UPDATE vw_simple
SET name='Karim'
WHERE id=1;
```

DELETE FROM VIEW

```
SQL
DELETE FROM vw_simple
WHERE id=1;
```

Rules of Updatable VIEW

A view becomes difficult/non-updatable if it contains:

- GROUP BY
- DISTINCT
- Aggregate Functions
- UNION
- TOP
- JOIN (sometimes)

WITH CHECK OPTION

Prevents inserting invalid rows through view.

Example

```
SQL
CREATE VIEW vw_newyork AS
SELECT *
FROM Salesman
WHERE city='New York'
WITH CHECK OPTION;
```

Now you cannot insert city='Paris' through this view.

SCHEMABINDING VIEW

Prevents changing base table structure.

Example

```
SQL
CREATE VIEW vw_bind
WITH SCHEMABINDING
AS
SELECT id,name
FROM dbo.Student;
```

Now table columns cannot be dropped easily.

Indexed VIEW

Stores result physically for faster performance.

Example

```
SQL
CREATE VIEW vw_index
WITH SCHEMABINDING
AS
SELECT id,name
FROM dbo.Student;
GO

CREATE UNIQUE CLUSTERED INDEX idx_view
ON vw_index(id);
```

Advantages of VIEW

Advantage	Description
Security	Hide sensitive data
Simplicity	Easy query writing
Reusability	Reuse query
Consistency	Same result every time
Abstraction	Users see limited data

Disadvantages of VIEW

Disadvantage	Description
Slow sometimes	Complex views reduce performance
Limited update	Some views not editable
Dependency	Table changes affect views

Difference Between TABLE and VIEW

TABLE	VIEW
Stores data physically	Does not store data normally
Has actual rows	Shows virtual rows
Takes storage	Very little storage
Faster sometimes	Depends on query

Multiple Important VIEW Examples

Example 1: View for High Salary

```
SQL
CREATE VIEW vw_high_salary AS
SELECT *
FROM Worker
WHERE salary > 50000;
```

Example 2: View Using Aggregate Function

```
SQL
CREATE VIEW vw_avg_salary AS
SELECT department,
AVG(salary) AS avg_salary
FROM Worker
GROUP BY department;
```

Example 3: View with JOIN

```
SQL
CREATE VIEW vw_emp_dept AS
SELECT e.name,
       d.department_name
FROM Employee e
JOIN Department d
ON e.dept_id=d.dept_id;
```

Execute/View Data

```
SQL
SELECT * FROM vw_emp_dept;
```

Rename View

SQL Server normally uses:

```
SQL
sp_rename 'old_view_name','new_view_name';
```

Best Practices

Use meaningful names

Good:

```
SQL
vw_customer_orders
```

Bad:

```
SQL
v1
```

Avoid SELECT *

Bad:

```
SQL
SELECT *
```

Good:

```
SQL
SELECT id,name
```

Keep views simple if possible

Complex nested views reduce performance.

Complete Example in One Code

```
SQL
CREATE TABLE Employee(
    id INT,
    name VARCHAR(50),
    salary INT,
    department VARCHAR(50)
);

INSERT INTO Employee VALUES
(1,'Rahim',50000,'CSE'),
(2,'Karim',60000,'EEE');

CREATE VIEW vw_employee AS
SELECT id,name,salary
FROM Employee
WHERE salary > 55000;

SELECT * FROM vw_employee;

ALTER VIEW vw_employee AS
SELECT id,name,department
FROM Employee;

SELECT * FROM vw_employee;

DROP VIEW vw_employee;
```

Interview Viva Questions

Q1: What is a View?

A virtual table based on SQL query.

Q2: Does View store data?

Normally no.

Q3: Can we update a View?

Yes, simple views can be updated.

Q4: Difference between View and Table?

Table stores data physically, view stores query only.

Q5: Why use Views?

Security, simplicity, reusability.

now give me full details about indexing

SQL Server INDEXING Full Details

What is INDEX?

An **INDEX** in SQL Server is a database object used to **speed up data searching and retrieval**.

It works like a **book index**.

Example:

- Without index → SQL Server checks every row.
- With index → SQL Server directly finds data faster.

Why We Use INDEX

Main Purposes

Purpose	Description
Faster SELECT	Speeds up searching
Faster WHERE	Quickly finds rows
Faster JOIN	Improves join performance
Faster ORDER BY	Sorting becomes faster
Faster GROUP BY	Grouping becomes faster

Real Example

Suppose a table has 1 million rows.

Without index:

```
SQL
SELECT * FROM Worker
WHERE WORKER_ID = 100;
```

SQL Server scans all rows.

With index:

- SQL Server directly jumps to the row.

Types of INDEX in SQL Server

Index Type	Description
Clustered Index	Sorts actual table data
Non-Clustered Index	Separate structure pointing to data
Unique Index	Prevents duplicate values
Composite Index	Uses multiple columns
Primary Key Index	Automatically created
Full Text Index	Used for text searching
Filtered Index	Index for specific rows
XML Index	For XML data
Columnstore Index	For analytics/big data

1. CLUSTERED INDEX

Definition

A clustered index physically arranges table data.

A table can have only ONE clustered index.

Example

```
SQL
CREATE TABLE Student(
  id INT PRIMARY KEY,
  name VARCHAR(50)
);
```

Primary key automatically creates clustered index usually.

Create Clustered Index

```
SQL
CREATE CLUSTERED INDEX idx_student_id
ON Student(id);
```

How Clustered Index Works

Data is physically sorted:

1
2

3
4
5

Searching becomes faster.

Advantages

- Very fast searching
- Good for range queries
- Faster sorting

Disadvantages

- Insert slower
- Update slower
- Only one clustered index allowed

2. NON-CLUSTERED INDEX

Definition

Creates separate structure storing indexed columns + pointers.

Table data itself remains unchanged.

Example

```
SQL
CREATE NONCLUSTERED INDEX idx_student_name
ON Student(name);
```

How Non-Clustered Works

Index stores:

Rahim → Row Address
Karim → Row Address

Advantages

- Faster search
- Multiple allowed
- Good for frequently searched columns

Disadvantages

- Uses extra storage
- Insert/update slower

3. UNIQUE INDEX

Prevents duplicate values.

Example

```
SQL
CREATE UNIQUE INDEX idx_email
ON Employee(email);
```

Duplicate email not allowed.

4. COMPOSITE INDEX

Uses multiple columns.

Example

```
SQL
CREATE INDEX idx_name_dept
ON Worker(FIRST_NAME,DEPARTMENT);
```

Useful for:

```
SQL
SELECT *
FROM Worker
WHERE FIRST_NAME='Rahim'
AND DEPARTMENT='CSE';
```

5. PRIMARY KEY INDEX

Primary key automatically creates unique index.

Example

```
SQL
CREATE TABLE Employee(
id INT PRIMARY KEY,
```



```
);
name VARCHAR(50)
```

CREATE INDEX Syntax

Basic Syntax

```
SQL
CREATE INDEX index_name
ON table_name(column_name);
```

DROP INDEX

Delete index.

Syntax

```
SQL
DROP INDEX index_name
ON table_name;
```

Example

```
SQL
DROP INDEX idx_student_name
ON Student;
```

View Existing Indexes

```
SQL
sp_helpindex 'Student';
```

ALTER INDEX

Used for rebuild or reorganize.

Rebuild Index

```
SQL
ALTER INDEX idx_student_name
ON Student
REBUILD;
```

Reorganize Index

```
SQL
ALTER INDEX idx_student_name
ON Student
REORGANIZE;
```

Clustered vs Non-Clustered Index

Clustered	Non-Clustered
Physically sorts data	Separate structure
One only	Multiple allowed
Faster	Slightly slower
Large storage changes	Less storage impact

When to Use INDEX

Good columns for indexing:

- Frequently searched columns
- WHERE clause columns
- JOIN columns
- ORDER BY columns
- GROUP BY columns

When NOT to Use INDEX

Avoid indexing:

- Small tables
- Frequently updated columns
- Columns with many duplicate values

Example Without Index

```
SQL
SELECT *
FROM Worker
WHERE salary = 50000;
```

Full table scan occurs.

Example With Index

```
SQL
```

```
CREATE INDEX idx_salary
ON Worker(salary);
```

Now search becomes faster.

Composite Index Order Important

```
SQL
CREATE INDEX idx_name_city
ON Customer(name,city);
```

Works best for:

```
SQL
WHERE name='Rahim'
```

or

```
SQL
WHERE name='Rahim' AND city='Dhaka'
```

But not efficient for only:

```
SQL
WHERE city='Dhaka'
```

Filtered Index

Indexes only specific rows.

Example

```
SQL
CREATE INDEX idx_active
ON Employee(status)
WHERE status='Active';
```

Full Text Index

Used for searching large text.

Example

```
SQL
CONTAINS(description,'database')
```

Index Fragmentation

Over time indexes become fragmented.

This reduces performance.

Fix Fragmentation

Reorganize

```
SQL
ALTER INDEX ALL
ON Employee
REORGANIZE;
```

Rebuild

```
SQL
ALTER INDEX ALL
ON Employee
REBUILD;
```

How SQL Server Uses INDEX

SQL Server Query Optimizer decides:

- Use index
- Or scan table

Execution Plan

Shortcut in SSMS:

CTRL + M

Shows query performance.

Important System Views

See all indexes

```
SQL
SELECT *
FROM sys.indexes;
```

See Index Usage

```
SQL
SELECT *
FROM sys.dm_db_index_usage_stats;
```

Advantages of INDEX

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Advantage	Description
Faster SELECT	Quick retrieval
Faster JOIN	Better join speed
Faster Sorting	ORDER BY improved
Better Performance	Large table optimization

Disadvantages of INDEX

Disadvantage	Description
Extra storage	Index uses memory
Slow INSERT	Index must update
Slow UPDATE	Extra maintenance
Slow DELETE	Index changes required

Best Practices

1. Index Important Columns Only

Good:

```
SQL
salary
employee_id
email
```

2. Avoid Too Many Indexes

Too many indexes slow INSERT/UPDATE.

3. Use Composite Index Carefully

Column order matters.

4. Rebuild Fragmented Indexes

Maintains performance.

Complete Example in One Code

```
SQL
CREATE TABLE Worker(
    WORKER_ID INT,
    FIRST_NAME VARCHAR(50),
    DEPARTMENT VARCHAR(50),
    SALARY INT
);

INSERT INTO Worker VALUES
(1,'Rahim','CSE',50000),
(2,'Karim','EEE',60000),
(3,'Sakib','BBA',45000);

-- Non Clustered Index
CREATE INDEX idx_salary
ON Worker(SALARY);

-- Composite Index
CREATE INDEX idx_name_dept
ON Worker(FIRST_NAME,DEPARTMENT);

-- Unique Index
CREATE UNIQUE INDEX idx_worker_unique
ON Worker(WORKER_ID);

-- Query
SELECT *
FROM Worker
WHERE SALARY=50000;

-- Show indexes
sp_helpindex 'Worker';

-- Drop index
DROP INDEX idx_salary
ON Worker;
```

Interview Viva Questions

Q1: What is Index?

Used to speed up data retrieval.

Q2: Difference between Clustered and Non-Clustered Index?

Clustered	Non-Clustered
Sorts actual data	Separate structure
One only	Many allowed

Q3: Why index improves performance?

Because SQL Server avoids full table scan.

Q4: Can index slow database?

Yes, INSERT/UPDATE/DELETE become slower.

Q5: Which columns should be indexed?

Frequently searched columns.

now give me full about cascading

SQL Server Cascading Full Details

What is Cascading?

Cascading means automatic action between related tables using **Foreign Key constraints**.

When parent table data changes or deletes, child table automatically updates or deletes depending on cascade rule.

Why We Use Cascading

Suppose:

- Parent Table → Departments
- Child Table → Employees

If a department is deleted:

- Should employee records remain?
- Or automatically delete?

Cascading controls this behavior.

Foreign Key Refresher

A foreign key connects two tables.

Example

```
SQL
CREATE TABLE Department(
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);

SQL
CREATE TABLE Employee(
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID)
REFERENCES Department(DeptID)
);
```

Here:

- Department → Parent table
- Employee → Child table

Types of Cascading Actions

Action	Meaning
ON DELETE CASCADE	Delete child rows automatically
ON UPDATE CASCADE	Update child rows automatically
ON DELETE SET NULL	Set foreign key NULL
ON DELETE SET DEFAULT	Set default value
ON DELETE NO ACTION	Prevent delete
ON DELETE RESTRICT	Restrict delete

1. ON DELETE CASCADE

When parent row deletes, child rows automatically delete.

Example

Parent Table

```
SQL
CREATE TABLE Departments(
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);
```

Child Table

SQL

```
CREATE TABLE Staff(
  StaffID INT PRIMARY KEY,
  StaffName VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID)
    REFERENCES Departments(DeptID)
    ON DELETE CASCADE
);
```

Insert Data

```
SQL
INSERT INTO Departments VALUES
(1,'CSE'),
(2,'EEE');
```

```
SQL
INSERT INTO Staff VALUES
(101,'Rahim',1),
(102,'Karim',1),
(103,'Sakib',2);
```

Delete Parent Row

```
SQL
DELETE FROM Departments
WHERE DeptID=1;
```

Automatically deletes:

- Rahim
- Karim

Result

Only Sakib remains.

2. ON UPDATE CASCADE

If parent primary key changes, child foreign keys automatically update.

Example

```
SQL
CREATE TABLE Clients(
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(50)
);

SQL
CREATE TABLE Orders(
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  FOREIGN KEY(CustomerID)
    REFERENCES Clients(CustomerID)
    ON UPDATE CASCADE
);
```

Insert Data

```
SQL
INSERT INTO Clients VALUES
(1,'Rahim');
```

```
SQL
INSERT INTO Orders VALUES
(1001,1);
```

Update Parent Key

```
SQL
UPDATE Clients
SET CustomerID=5
WHERE CustomerID=1;
```

Automatically:

Orders.CustomerID becomes 5

3. ON DELETE SET NULL

When parent row deletes, child foreign key becomes NULL.

Example

```
SQL
CREATE TABLE Employee(
  EmpID INT PRIMARY KEY,
  DeptID INT NULL,
  FOREIGN KEY(DeptID)
    REFERENCES Department(DeptID)
    ON DELETE SET NULL
);
```

Result

Delete department:

- Employee still exists
- DeptID becomes NULL

4. ON DELETE SET DEFAULT

Sets default value after parent deletion.

Example

```
SQL
CREATE TABLE Employee(
  EmpID INT PRIMARY KEY,
  DeptID INT DEFAULT 0,
  FOREIGN KEY(DeptID)
  REFERENCES Department(DeptID)
  ON DELETE SET DEFAULT
);
```

5. NO ACTION / RESTRICT

Prevents parent deletion if child exists.

Example

```
SQL
CREATE TABLE Orders(
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  FOREIGN KEY(CustomerID)
  REFERENCES Clients(CustomerID)
  ON DELETE NO ACTION
);
```

What Happens?

If child rows exist:

```
SQL
DELETE FROM Clients
WHERE CustomerID=1;
```

Error occurs.

Difference Between NO ACTION and CASCADE

CASCADE	NO ACTION
Deletes child rows	Prevents deletion
Automatic cleanup	Protects integrity

Composite Key Cascading

Works with multiple columns.

Parent Table

```
SQL
CREATE TABLE Projects(
  ProjectID INT,
  DeptCode VARCHAR(10),
  PRIMARY KEY(ProjectID,DeptCode)
);
```

Child Table

```
SQL
CREATE TABLE ProjectTasks(
  TaskID INT PRIMARY KEY,
  ProjectID INT,
  DeptCode VARCHAR(10),
  FOREIGN KEY(ProjectID,DeptCode)
  REFERENCES Projects(ProjectID,DeptCode)
  ON DELETE CASCADE
);
```

Alter Existing Table for Cascade

Add Foreign Key Later

```
SQL
ALTER TABLE Orders
ADD CONSTRAINT fk_customer
FOREIGN KEY(CustomerID)
REFERENCES Clients(CustomerID)
ON DELETE CASCADE;
```

Drop Foreign Key

Find Constraint Name

```
SQL
sp_help Orders;
```

Drop Constraint

```
SQL
ALTER TABLE Orders
DROP CONSTRAINT fk_customer;
```

Important Rules

Parent Table Must Have

- PRIMARY KEY
- or
- UNIQUE KEY

Data Types Must Match

Correct:

`INT → INT`

Wrong:

`INT → VARCHAR`

Advantages of Cascading

Advantage	Description
Automatic maintenance	Less manual work
Maintains integrity	Prevents orphan records
Cleaner database	Related rows handled automatically
Saves time	Easier management

Disadvantages of Cascading

Disadvantage	Description
Risky delete	Large accidental delete possible
Hard debugging	Automatic actions hidden
Performance issue	Large cascades slower

Real Life Example

Parent

Customer

Child

Orders

Delete customer:

- Should orders delete too?

Use:

SQL
`ON DELETE CASCADE`

View Foreign Keys

SQL
`SELECT *`
`FROM sys.foreign_keys;`

Full Working Example

```
SQL
CREATE TABLE Department(
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);

CREATE TABLE Employee(
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID)
    REFERENCES Department(DeptID)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

INSERT INTO Department VALUES
(1,'CSE'),
(2,'EEE');

INSERT INTO Employee VALUES
(101,'Rahim',1),
(102,'Karim',1);

SELECT * FROM Department;
SELECT * FROM Employee;

-- Update parent key
UPDATE Department
SET DeptID=5
WHERE DeptID=1;

-- Delete parent
DELETE FROM Department
WHERE DeptID=5;

SELECT * FROM Employee;
```

Output Explanation

After UPDATE:

Employee.DeptID automatically becomes 5

After DELETE:

Related employees automatically deleted

Cascade Flow Diagram

Parent Table
↓
Foreign Key
↓
Child Table

With CASCADE:

Parent change/delete
↓
Child automatically changes/deletes

Best Practices

Use CASCADE Carefully

Large delete may remove huge data.

Use NO ACTION for Important Data

Safer for financial systems.

Always Backup Important Database

Before large cascading operations.

Interview Viva Questions

Q1: What is Cascading?

Automatic action between parent and child tables.

Q2: What is ON DELETE CASCADE?

Deletes child rows automatically.

Q3: What is ON UPDATE CASCADE?

Updates foreign keys automatically.

Q4: Why use cascading?

Maintains referential integrity.

Q5: Can cascading be dangerous?

Yes, accidental deletes may remove large data.